Hamava — Project Accounts & Services Notebook

Purpose of this file: a single place to see every external service this project depends on, why it exists, and where its actual credentials live. **This file intentionally contains NO real passwords, keys, or secrets** — only account emails, URLs, and pointers to where the real values are stored (`backend/.env`, which is gitignored and never shared).
Keep this file somewhere safe (not a public folder, not shared casually) — even without secrets in it, it's a map of what accounts exist for this project.

☁ Cloud accounts (require login to manage)

1. Neon — Primary Database

What it is: PostgreSQL, the main database for the entire app
Website: neon.tech
What's stored there: every real piece of app data — user accounts, profiles, photos metadata, swipes, matches, chat messages, reports, audit logs
Plan: Free tier (no card required)
Where credentials live: `backend/.env` → `DATABASE_URL` (pooled connection) and `DIRECT_URL` (used for migrations)
Why it exists: every feature built so far (auth, profile, swipe, chat, report/block, settings) writes to this database
If this account is lost/deleted: the entire app's data would be lost — this is the single most important account in the list

2. Upstash — Redis (Real-Time Messaging Layer)

What it is: Redis, used as the pub/sub backbone for real-time chat delivery
Website: upstash.com
What's stored there: no permanent user data — only transient real-time message-routing state
Plan: Free tier (no card required)
Where credentials live: `backend/.env` → `REDIS_URL`
Why it exists: lets the chat feature deliver messages instantly and lets the backend scale to multiple servers later without breaking real-time delivery
If this account is lost: chat's real-time push would stop working, but no permanent data would be lost (messages themselves are safely in Neon)

3. OpenAI — Content Moderation

What it is: the Moderation API (model: `omni-moderation-latest`), used to automatically screen text (bios, chat messages) and photos for unsafe content
Website: platform.openai.com (**not** the regular ChatGPT chat.openai.com — this is the separate developer/API side)
Account used: same login as your regular ChatGPT account
Billing status: payment method added, $5 prepaid credit purchased to activate the account; **auto-reload is OFF** (deliberately, so it never recharges itself automatically)
Where credentials live: `backend/.env` → `OPENAI_API_KEY`
Why it exists: required for Feature 8 (Real Content Moderation) — flags unsafe bios, messages, and photos before they're visible to other users

If this account is lost: content moderation would stop working; the app's production-safety guard is designed to refuse to start in production without a working moderation provider, so this would block launch, not silently degrade

4. Cloudflare R2 — Photo Storage (Production)

What it is: S3-compatible cloud object storage, used to store user-uploaded profile photos in production
Website: dash.cloudflare.com
Bucket name: `hamava`
Plan: Free tier — 10 GB storage, 1 million Class A operations/month, 10 million Class B operations/month included; card on file for any usage beyond that (no hard spending cap exists on Cloudflare — see note below)
Where credentials live: `backend/.env` → check for `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_ENDPOINT`, `R2_BUCKET_NAME` (or similarly named variables — confirm exact names in the file itself)
Why it exists: the production-grade replacement for photo storage, verified to actually work with the real photo-processing pipeline (upload → virus scan → content moderation → storage → time-limited access link)
Important reminder: because these credentials were shared in a chat conversation during setup, they should be **rotated (regenerated) in the Cloudflare dashboard** once initial verification is complete, as a precaution
Billing note: unlike OpenAI, Cloudflare has no automatic hard spending cap — only email alerts. A budget alert should be confirmed active in Cloudflare's dashboard (Billing → Billable Usage) as an extra safety net

💻 Local software (installed on this computer only — not online accounts)

5. MinIO — Local Development Storage

What it is: a local, S3-compatible storage server, used only for development/testing before Cloudflare R2 was verified
Where it runs: locally on this PC (`C:\MinIO\data`, port 9010) — not accessible from the internet, not an online account
Why it exists: let photo-storage code be built and tested against a real S3-protocol server without needing a cloud account during early development

6. ClamAV — Local Antivirus Scanning

What it is: a real antivirus engine (`clamd`), used to scan every uploaded photo for malware before it's accepted
Where it runs: locally on this PC, as a background service (port 3310)
Why it exists: required so photo uploads are never accepted without a genuine malware scan — this is real, not a placeholder, and was verified with an industry-standard test file (EICAR)
Production note: documented in `docs/architecture.md` that this needs to run as its own dedicated service (not on the same machine as the API) once deployed for real, with its own monitoring

🔧 Development tool (not a service the app depends on, but part of how it's built)

7. Claude Code

What it is: the AI coding assistant used to actually write, test, and verify all the backend code described above

Where it runs: on this PC, inside its own project window

Not a credential concern: doesn't store app secrets itself beyond what's already in `backend/.env` on this machine

🔒 Security reminders

Never put real secret values in this file — if you ever need to record an actual key/password somewhere, use a dedicated password manager, not a plain document.

The real secrets live in one place: `backend/.env` on this computer, which is excluded from version control (`.gitignore`) and never uploaded anywhere by the code.

Rotate (regenerate) any credential that was ever typed into a chat conversation — this applies to the Cloudflare R2 keys and the earlier Redis/Upstash password, both of which were shared this way during setup.

Two legal-blocked services are still pending (not yet real accounts): an SMS/OTP provider (waiting on sanctions/export-control legal review) and a liveness-verification vendor (waiting on a completed DPIA) — these will be added to this notebook once selected.